

Intermediate Protocol

Software Engineering 2 Labor
SS 2026

Subject: Tour-Planner

Team name/team members:
Sabic Kenan, if24b080@technikum-wien.at
Schwarz Carl, if25b237@technikum-wien.at

Supervisor: ***DI. Berger Wolfgang***

Einleitung

Für das dieses Semester war das frontend einer Tourenplanungs-Applikation über Angular20 zu entwerfen.

User sollen sich über diese Applikation registrieren können und touren mit einem Start und Ziel-Punkt sowie Titel, Beschreibung und Transporttyp einspeichern.

Zusätzlich soll der zu zurücklegende Weg, sowie die benötigte Zeit von dem Leaflet-Service abgerufen werden und in die Tourenansicht einfließen.

Zu diesen Touren können Logs mit Kommentar, Bewertung, Schwierigkeitsgrad, Zeitstempel und der benötigten zeit zu einer Tour hinzugefügt werden können. Diese sollen jedoch nur für den Ersteller sichtbar sein.

Die Entwicklung sowie Versionskontrolle verlief über GitHub. Das Programm ist über folgenden Link erreichbar:

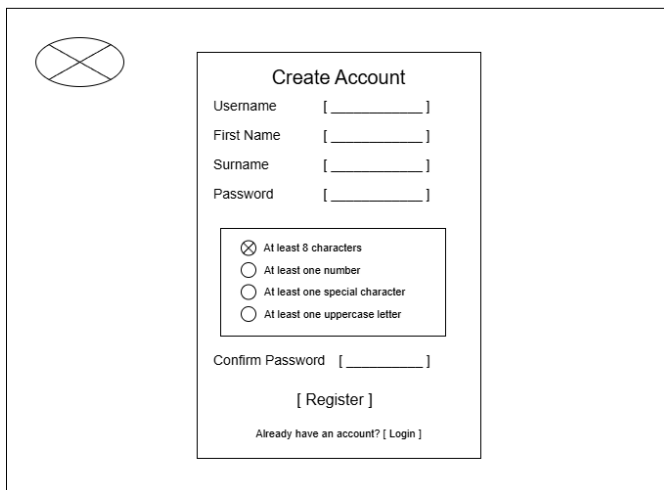
Git Link: <https://github.com/tempestkny/TourPlanner.git>

Wireframes

Register

Der User soll über die Applikation einen Account erstellen können, welcher seine Email-Adresse und Nutzernamen beinhaltet und durch ein Passwort geschützt ist.

Register



The wireframe shows a registration form titled "Create Account". It contains the following elements:

- A placeholder icon (a circle with an 'X') in the top left corner.
- Input fields for "Username", "First Name", "Surname", "Password", and "Confirm Password".
- A section for password requirements with four radio buttons:
 - ☒ At least 8 characters
 - ☐ At least one number
 - ☐ At least one special character
 - ☐ At least one uppercase letter
- A "[Register]" button.
- A link "Already have an account? [Login]" at the bottom.

Abbildung 1: Wireframe Registrier-Seite

Tour-Listenansicht

Für die Listenansicht wurden zwei Designs zur Erwägung gezogen.

Die erste Variante zielt auf Verständnis ab, indem möglichst viel Information in den Einträgen angezeigt werden, wodurch der Nutzer einen groben Überblick über die Details der Touren erhält.

Die zweite Variante zielt auf Nutzungsgeschwindigkeit ab, indem die angezeigte Information auf ein Minimum reduziert wird, und somit mehr Touren auf einer Fläche für den Nutzer zur Auswahl stehen.

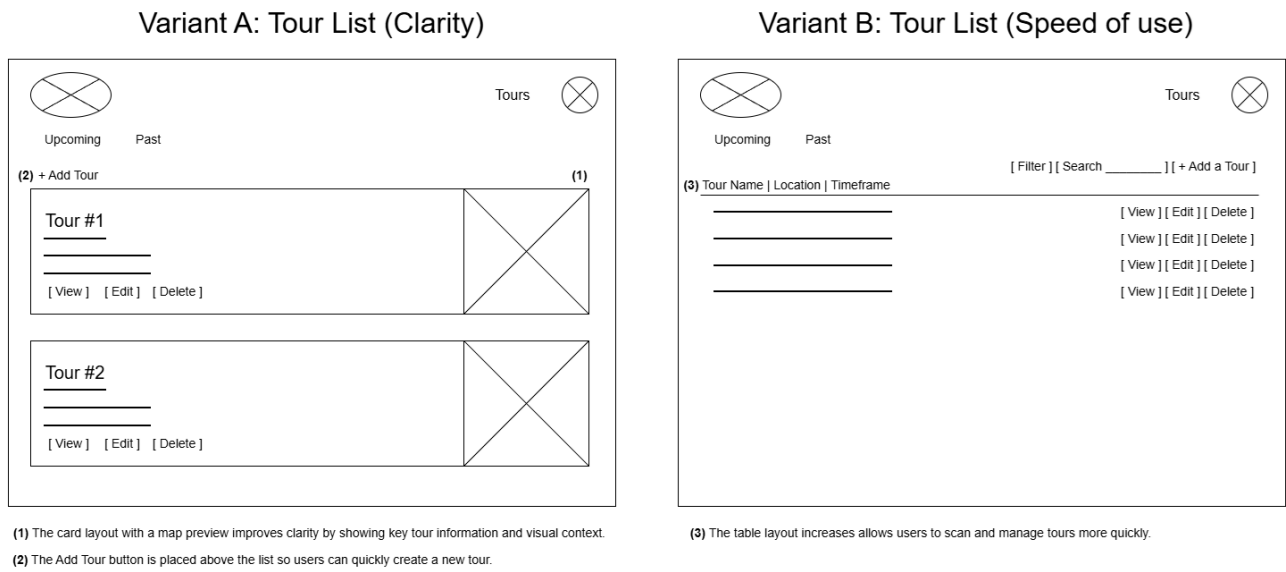


Abbildung 2: Tour-Listenansicht Varianten

Es wurde sich für das Design der UI für Variante A entschieden, da die Übersicht und Menge an Information die Debug-Tests erleichtert

Tour-Detailansicht

Von der Listenansicht soll auf die Tour-Detailansicht gewechselt werden. Die Tour-Detailansicht zeigt die Attribute der Tour an (Beschreibung, Start, Ziel, Karte, etc)

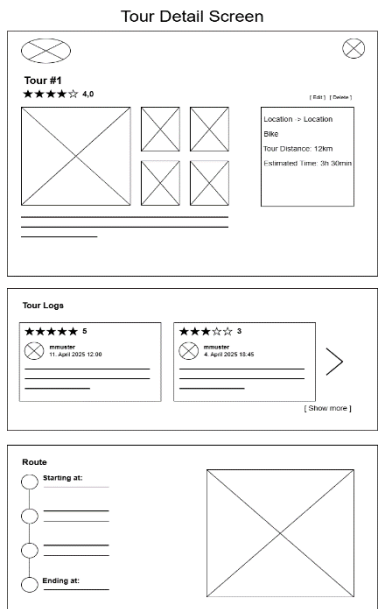


Abbildung 3: Tour Detailansicht

Tour-Logs

Die Tour-Logs können zu jeder Tour erstellt und über die Tour-Listenansicht aufgerufen werden.

Tour Logs

Overall rating
★★★★☆ 4,0 / 5

[Filter] [Search _____] [+ Add a Log]

★★★★★ 5

 mmuster
11. April 2025 12:00

[Edit]
[Delete]

Difficulty: ★★★★★
Total Distance: 12km
Total Time: 2h 30 min

★★★★☆ 3

 mmuster
4. April 2025 18:40

[Edit]
[Delete]

Difficulty: ★★★★★
Total Distance: 12km
Total Time: 3h 30 min

Abbildung 4: Tour Logs Liste

System Architektur

Komponenten

Tour-Shell

Um die Nutzungsgeschwindigkeit zu erhöhen, wurde entschlossen, dass die Tourplanner seite in zwei Teile aufgeteilt wird. Die linke Seite zeigt die Tour-Listenansicht, während die rechte Seite zur Anzeige der Tour-Details, Tour-Logs und deren Verwaltung zuständig ist.

Für anzeige beider Seiten ist die Tour Shell zuständig, die in ihrer Template Datei zu den jeweiligen Templates der Komponenten referenziert und über eine state-machine steuert, welche zu diesem Zeitpunkt angezeigt werden soll.

Zusätzlich übernimmt die Shell die Rolle eines Mediators, der die von den Komponenten ausgelösten Events entgegennimmt und in entsprechende Aktionen (z. B. Anzeigen, Bearbeiten oder Löschen eines Logs) übersetzt.

```

1 <div class="split-view">
2   <div class="left">
3     <router-outlet name="list"></router-outlet>
4     <app-tour-list
5       (selectTour)="onSelectTour($event)"
6       (createTour)="onCreateTour()"
7       (viewLogList)="onViewLogList($event)"
8       (createLog)="onCreateLog($event)"
9     >
10    </app-tour-list>
11  </div>
12
13  <div class="right">
14    <!--Event Handling of which page the shell should display on the right side-->
15    @switch (mode) {
16      @case('detail'){
17        <app-tour-detail
18          [tour]="selectedTour"
19          (editTour)="onEditTour($event)"
20          (deleteTour)="onDeleteTour($event)">
21        </app-tour-detail>
22      }
23
24      @case('edit'){
25        <app-tour-edit
26          [tour]="selectedTour"
27          (saved)="onEditSaved($event)"

```

Abbildung 4: Codeausschnitt der Shell-Template

Tour-Ansicht

Tour-List

Die Tour-List ist die primäre Navigationskomponente für die Touren. Sie bildet den linken Bereich der Anwendung und dient als Ausgangspunkt für alle weiteren Aktionen wie Anzeigen, Bearbeiten oder Löschen einer Tour.

Touren werden über Einträge in der Liste angezeigt, welche von dem User für nähere Information angeklickt werden können.

Die Verwaltung der Touren läuft über den Tour-List-Service, die Touren sind in einer reaktiven BehaviourSubject-Liste gespeichert. Jede Änderung an dieser Liste wird über das Observable logs\$ an die Tour-List-Komponente weitergegeben, wodurch sich die Listenansicht automatisch aktualisiert.

```

// Uses a BehaviourSubject to Store the Tours
// Subscribers get the value of the Subject immediately
// Updates on the BehaviourSubject will trigger the other subscribers to update theirs too
export class TourListService {
  private toursSubject = new BehaviorSubject<TourItemInterface[]>([]);
  tours$ = this.toursSubject.asObservable();

  get tours(): TourItemInterface[] {
    return this.toursSubject.value;
  }

  addTour(tour: TourItemInterface) {
    // creates a new Array which has the new tour at the end
    const updated = [...this.toursSubject.value, tour];
    this.toursSubject.next(updated);
  }

  updateTour(originalTour: TourItemInterface, updatedTour: TourItemInterface): void {
    const updated = this.toursSubject.value.map(tour =>
      | tour === originalTour ? updatedTour : tour
    );
  }
}

```

Abbildung 4: Tour-List-Service.ts codeausschnitt (BehaviourSubject)

Tour-Creation

Die Tour-Creation-Komponente stellt ein Formular zur Eingabe der Tourdaten bereit.

Bevor eine Tour erstellt werden kann, überprüft die Komponente mithilfe des Leaflet-Services, ob sowohl Start- als auch Zielort gültig sind. Erst nach erfolgreicher Validierung wird der ‚Create Tour‘-Button aktiviert.

Beim Erstellen löst die Komponente zwei Events aus: Das erste übermittelt die Tourdaten an den Tour-List-Service, der die neue Tour in die zentrale Liste einträgt. Das zweite Event informiert die Shell über den erfolgreichen Vorgang, sodass diese in die Detailansicht der neu erstellten Tour wechseln kann.

Tour-View

Die Tour-View stellt die Detailansicht einer ausgewählten Tour dar und wird über die Tour-Liste aufgerufen.

Sie erhält die aktuelle Tour über ein Input-Property von der Shell und zeigt die vorhandenen Tourdaten sowie eine Vorschau der zugehörigen Logs an. Benutzeraktionen wie das Bearbeiten, Löschen oder Anzeigen aller Logs werden über Output Events an die Shell weitergegeben.

Die Shell übernimmt die Zustandsänderung und steuert den weiteren UI Modus. Die Komponente selbst enthält keine Businesslogik und dient ausschließlich der Darstellung und Event-Weiterleitung.

Tour-Edit und -Delete

Die Tour-Edit-Komponente ermöglicht das Bearbeiten einer bestehenden Tour.

Beim Öffnen wird die ausgewählte Tour über ein Input-Property übergeben und in ein lokales Modell übernommen, sodass Änderungen zunächst nur lokal erfolgen. Beim Speichern sendet die Komponente ein Event mit den aktualisierten Daten an die Shell, welche den Tour-List-Service aktualisiert und zur Detailansicht zurückwechselt.

Das Löschen einer Tour wird aus der Detailansicht ausgelöst und ebenfalls über ein Event an die Shell gemeldet. Die Shell entfernt die Tour über den Tour-List-Service aus der zentralen Liste.

Beide Komponenten enthalten keine eigene Businesslogik und dienen ausschließlich der Darstellung und Event Weiterleitung.

Tour-Logs

Create-Tour-Log

Die Create-Tour-Log-Komponente dient zur Erstellung eines neuen Tour-Logs für die aktuell ausgewählte Tour. Sie stellt ein Formular bereit, in dem der Benutzer die relevanten Log-Daten wie Datum, zurückgelegte Distanz und Zeit eintragen kann.

Die Komponente führt grundlegende Validierungen durch, ob Distanz und Zeit gültige numerische Werte sind.

Nach erfolgreicher Eingabe erzeugt sie ein vollständiges Tour-Log-Objekt und sendet dieses über ein Output-Event an die Shell. Die Shell fügt das neue Log anschließend über den Log-List-Service in die zentrale Log-Liste ein und wechselt zurück in die entsprechende Ansicht.

Die Create-Tour-Log-Komponente enthält keine eigene Businesslogik und führt keine

Datenmutationen durch; sie ist ausschließlich für die Datenerfassung und Event-Kommunikation zuständig.

Tour-Log-List

Die Tour-Log-List ist die Navigationskomponente für die Logs der aktuell ausgewählten Tour und wird rechts neben der Tour-Liste angezeigt. Sie zeigt alle vorhandenen Logs in einer scrollbaren Liste an und meldet die Auswahl eines Logs über ein Event an die Shell, welche anschließend in die entsprechende Detailansicht wechselt.

Die Log-Daten werden zentral im Log-List-Service verwaltet, der sie in einem BehaviorSubject speichert und bei Änderungen automatisch an die Tour-Log-List weitergibt. Die Komponente selbst enthält keine Businesslogik und dient ausschließlich der Darstellung und Event-Weiterleitung.

View-Tour-Log

Die View-Tour-Log zeigt die vollständigen Details eines ausgewählten Tour-Logs an und ist über die Tour-Log-List erreichbar.

Sie dient ausschließlich der Darstellung und bietet dem Benutzer Optionen zum Bearbeiten oder Löschen des Logs. Aktionen wie „Edit“ oder „Delete“ werden nicht direkt ausgeführt, sondern über Output-Events an die Shell gemeldet.

Die Shell entscheidet anschließend über den weiteren UI-Modus und führt die entsprechenden Operationen im Log-List-Service aus. Die Komponente selbst enthält keine Businesslogik und keine Datenhaltung.

Edit-Tour-Log

Die Edit-Tour-Log-Komponente ermöglicht das Bearbeiten eines bestehenden Tour-Logs.

Beim Öffnen wird das zu bearbeitende Log über ein Input-Property übergeben und in ein lokales Edit-Modell übernommen. Änderungen an den Feldern werden lokal gehalten, bis der Benutzer speichert.

Beim Speichern sendet die Komponente ein Event mit den aktualisierten Log-Daten an die Shell. Die Shell aktualisiert daraufhin den Log-List-Service und wechselt zurück in die Log-Listenansicht. Die Komponente führt keine eigenen Datenmutationen durch und enthält keine Businesslogik.